Bayesian Neural Networks for Image Classification

MTH6161: Neural Networks and Deep Learning Mini Project

Mohammad Rayyan Hameed

Student Number: 231172804

May 15, 2026

Abstract

In this project we investigate the use of Bayesian Neural Networks for the purposes of Image classification. Standard Neural Networks, whilst still very effective, only learn point estimates of weights meaning that they cannot provide any measure of predictive uncertainty. This can be a massive limitation especially in applications such as self driving cars or medical imaging, where some notion of uncertainty is of the utmost important. This project explores the use of Bayesian Neural Networks (BNN's) trained on the Fashion M-NIST dataset where each weight is represented by a variational posterior $q(\theta) = \mathcal{N}(\mu, \sigma^2)$ rather than some single fixed value. We then train the network with the goal of learning the true posterior distribution, using KL divergence to derive the ELBO(evidenced lower bound) and then use the ELBO as our loss function, so that at test time we can use this posterior to find our predictive posterior distribution, which can then allow us to generate uncertainty estimates. Performance is evaluated in terms of classification accuracy and Expected Calibration Error (ECE), with results compared against a standard neural network baseline of identical architecture. The BNN additionally provides decomposition of predictive uncertainty into epistemic and aleatoric components, a capability the baseline entirely lacks.

1 Motivation and Task description

As reliance on AI systems grows, quantifying the uncertainty of model predictions has become increasingly important. Standard neural networks produce point estimates of weights, yielding a single confident prediction with no measure of how certain that prediction is. Consider a self-driving car approaching a stop sign. If the network assigns 49 percent probability to the correct class and 51 percent to an incorrect one, the system acts on that prediction with no awareness of how close the decision was. In safety critical settings such as autonomous driving, medical diagnosis, and financial forecasting, this lack of uncertainty quantification is a massive limitation. Bayesian Neural Networks (BNNs) address this by replacing fixed weight estimates with probability distributions over weights, allowing the model to express uncertainty alongside its predictions. This project implements a BNN trained on the FashionMNIST dataset using Variational Inference, and evaluates both its classification performance and the quality of its uncertainty estimates. To contextualise performance, results are compared against a standard neural network baseline of identical architecture trained with cross entropy alone.

2 Methodology: Background Theory and Network Architecture/Training Strategy

2.1 Basics of Bayesian Statistics

We assume familiarity with the basics of Bayesian statistics. For a comprehensive introduction we refer the reader to Bishop [1]. We recall that the posterior distribution is given by Bayes theorem:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta) \cdot p(\theta)}{p(\mathcal{D})} \tag{1}$$

where $p(\theta)$ is the prior, $p(\mathcal{D}|\theta)$ is the likelihood, and $p(\mathcal{D})$ is the marginal likelihood. The posterior is what we wish to learn, but computing it exactly is intractable as $p(\mathcal{D}) = \int p(\mathcal{D}|\theta) \cdot p(\theta)\, d\theta$ requires integrating over all possible parameter configurations. This motivates the use of Variational Inference.

2.2 Variational Inference and the ELBO

Since $p(\theta|\mathcal{D})$ is intractable, we instead approximate it with a simpler variational distribution $q(\theta) = \mathcal{N}(\mu, \sigma^2)$. The goal is to find the μ and σ that make $q(\theta)$ as close as possible to the true posterior, measured by KL divergence $\mathrm{KL}(q(\theta)\,||\,p(\theta|\mathcal{D}))$. Minimising this directly requires knowing $p(\theta|\mathcal{D})$, which is intractable. It can be shown, as in [2], that this is equivalent to maximising the Evidence Lower Bound (ELBO):

$$\mathrm{ELBO} = \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \mathrm{KL}(q(\theta)\,||\,p(\theta)) \tag{2}$$

Crucially, the true posterior $p(\theta|\mathcal{D})$ has disappeared entirely and the ELBO only involves tractable quantities: the likelihood $p(\mathcal{D}|\theta)$, the variational posterior $q(\theta)$, and the prior $p(\theta)$. In practice we minimise the negative ELBO:

$$\mathcal{L} = \underbrace{H(y, \hat{y})}_{\text{cross entropy}} + \beta \cdot \frac{1}{N} \underbrace{\mathrm{KL}(q(\theta)\,||\,p(\theta))}_{\text{regularisation}} \tag{3}$$

where N is the dataset size and β is a KL annealing weight discussed in Section 2.4. Having established the variational inference framework, we now describe how this is applied to a Bayesian neural network.

2.3 Bayesian Neural Network Architecture

Now that we have established all the theory and background we can formally describe what our network will look like. We opt to use a four layer neural network (one input layer, two hidden layers and one output layer) that we train on the FashionMNIST dataset [4] (10 different clothing categories). The input layer is made up of 784 neurons, the first hidden layer has 200 neurons, the second hidden layer

has 80 neurons and the final output layer has 10 neurons. Unlike a regular neural network where we only have one weight matrix W and one bias vector b for each layer, we instead have two weight matrices W_μ, W_σ and two bias vectors b_μ, b_σ for each layer. This is because for each weight, rather than trying to learn one single point estimate, we want to find the optimal μ and σ for each weight. Meaning we need one weight matrix to hold all the μ's for a layer and another to hold all the σ's for the layer. We note that these two matrices are of an identical shape within each layer. In practice, rather than storing σ directly for each weight, we instead store a raw parameter ρ and compute σ via the softplus function:

$$\sigma = \log(1 + e^{\rho}) \tag{4}$$

This ensures σ remains strictly positive whilst maintaining smooth gradients throughout training. The standard deviation for weights and biases in each layer is computed this way, giving us the full variational posterior $q(\theta) = \mathcal{N}(\mu, \sigma^2)$ for every weight.

The forward pass through each hidden layer proceeds identically to a standard neural network. For layer l, the pre-activation and activation are computed as:

$$a^{(l+1)} = W^{(l)} x^{(l)} + b^{(l)}, \quad x^{(l+1)} = \text{ReLU}(a^{(l+1)}) \tag{5}$$

where $W^{(l)}$ is sampled from the variational posterior $q(W) = \mathcal{N}(W_\mu, \sigma^2)$ at each forward pass. The output layer uses softmax to produce a probability distribution over the 10 FashionMNIST classes:

$$y_i = \frac{\exp(a_i^{(L)})}{\sum_{j=1}^{10} \exp(a_j^{(L)})} \tag{6}$$

We choose a prior of $p(\theta) = \mathcal{N}(0, 4)$ with $\sigma_{\text{prior}} = 2.0$; a relatively uninformative prior that gives the posterior freedom to move away from zero and fit the data without imposing overly strong regularisation. A standard neural network of identical architecture is also trained as a baseline for comparison, replacing each BayesianLinear layer with a standard linear layer and removing the KL term from the loss, training with cross entropy alone. This allows us to directly assess the cost of uncertainty quantification in terms of predictive performance and calibration.

2.4 Training Time

At each training step, we perform a forward pass by sampling weights from the variational posterior $q(\theta)$ for each layer. Since sampling is a non-differentiable operation, gradients cannot flow directly through the sampling step to μ and ρ. To resolve this we employ the reparameterisation trick [2], expressing each weight sample as:

$$\theta = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \tag{7}$$

This separates the randomness into ε, which is independent of the learnable parameters, allowing gradients to flow through μ and σ during backpropagation.

The KL divergence between the variational posterior and the prior has a closed form for two Gaussians, which we exploit to avoid additional sampling:

$$\text{KL}\left(\mathcal{N}(\mu, \sigma^2) \,||\, \mathcal{N}(0, \sigma_{\text{prior}}^2)\right) = \log \frac{\sigma_{\text{prior}}}{\sigma} + \frac{\sigma^2 + \mu^2}{2\sigma_{\text{prior}}^2} - \frac{1}{2} \tag{8}$$

This is computed analytically for every weight and bias in each layer and summed to give the total KL contribution.

In practice we minimise the negative ELBO as defined in Equation 3 where N is the dataset size and β is a KL annealing weight. Dividing the KL by N ensures the mini-batch loss is an unbiased estimator of the full dataset ELBO regardless of batch size.To stabilise early training, we employ KL annealing. This is where we ramp up β linearly from 0 to 1 over the first 5 epochs. This allows the cross entropy term to dominate early on, letting the network fit the data before the KL regularisation

switches on. Without annealing, the KL term can collapse the posterior towards the prior before the network has learned anything meaningful.The network is trained using the Adam optimiser. Full training hyperparameters are summarised in Table 1.

Table 1: Training hyperparameters

Hyperparameter	**Value**	**Hyperparameter**	**Value**
Optimiser	Adam	Prior σ	2.0
Learning rate	10^{-3}	Hidden layers	2
Batch size	128	Hidden units	200, 80
Epochs	10	MC samples	50
KL warmup	5		

2.5 Testing Time

At test time we wish to obtain the posterior predictive distribution; a distribution over class predictions that accounts for uncertainty in the weights:

$$p(y^*|x^*, \mathcal{D}) = \int p(y^*|x^*, \theta) \cdot p(\theta|\mathcal{D})\, d\theta \tag{9}$$

This integral is intractable, so we approximate it using Monte Carlo sampling. Rather than a single forward pass with fixed weights, we draw $S = 50$ weight samples from the variational posterior $q(\theta)$, run each through the network, and average the resulting predictions:

$$p(y^*|x^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} p(y^*|x^*, \theta_s), \quad \theta_s \sim q(\theta) \tag{10}$$

The final class prediction is the argmax of the averaged softmax outputs. Importantly, this procedure also yields uncertainty estimates for free: the spread across the S predictions reflecting the model's uncertainty about that input.

We decompose the total predictive uncertainty into two components. Let $\bar{p} = \frac{1}{S} \sum_s p(y^*|x^*, \theta_s)$ denote the mean predicted distribution. The **predictive entropy** captures total uncertainty:

$$H[\bar{p}] = -\sum_c \bar{p}_c \log \bar{p}_c \tag{11}$$

The expected entropy captures aleatoric uncertainty which is essentially the irreducible noise in the data itself:

$$\mathbb{E}[H] = \frac{1}{S} \sum_{s=1}^{S} \left(-\sum_c p_c^{(s)} \log p_c^{(s)} \right) \tag{12}$$

The difference between these two quantities gives the epistemic uncertainty, captured by the mutual information:

$$\mathcal{I} = H[\bar{p}] - \mathbb{E}[H] \tag{13}$$

Epistemic uncertainty reflects uncertainty due to limited knowledge of the weights and can in principle be reduced with more training data, unlike aleatoric uncertainty which is irreducible.

Finally, we evaluate calibration using the Expected Calibration Error (ECE), which measures whether the model's confidence scores match its empirical accuracy. A perfectly calibrated model assigned 80% confidence should be correct 80% of the time. ECE is computed by binning predictions by confidence and measuring the weighted average gap between confidence and accuracy across bins.

3 Results and Analysis

3.1 Classification Performance

Table 2 summarises the classification performance of both models on the FashionMNIST test set.

Table 2: Classification results on FashionMNIST test set

Model	Test Accuracy	ECE
Baseline NN	88.2%	0.016
BNN (MC 50)	86.0%	0.019

The baseline achieves marginally higher accuracy (88.2% vs 86.0%), which is expected as the KL divergence term in the ELBO acts as regularisation, pulling the variational posterior towards the prior and preventing the BNN from fitting the data as freely as the baseline. The accuracy gap of 2.2% represents a modest cost for the uncertainty quantification capabilities the BNN provides.

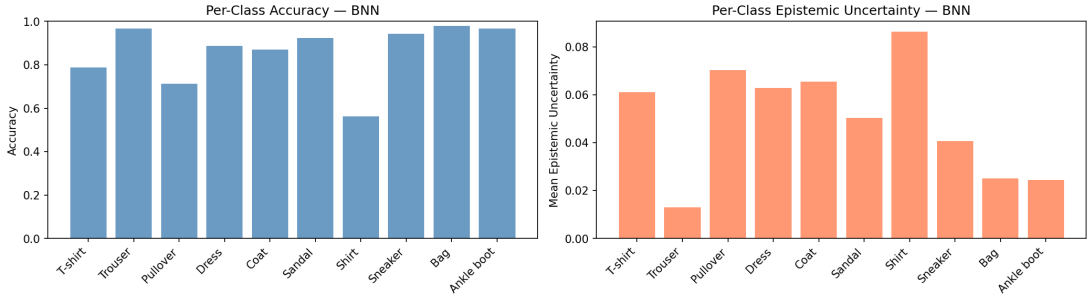


Figure 1: Per-class accuracy (left) and mean epistemic uncertainty (right) for the BNN on the FashionMNIST test set. Classes with lower accuracy tend to exhibit higher epistemic uncertainty, suggesting the model correctly identifies where it is uncertain.

Figure 1 shows the per-class accuracy breakdown for the BNN. Performance varies significantly across classes. The Trouser, Bag, Sneaker and Ankle Boot all achieve above 95% accuracy, whilst Shirt (57%) and Pullover (71%) are considerably lower. This is consistent with the known difficulty of FashionMNIST where visually similar classes such as Shirt, T-shirt, Pullover and Coat are frequently confused by the network, whilst structurally distinct classes such as Trouser and Bag are classified with high confidence. Notably, the per-class epistemic uncertainty closely mirrors the per-class accuracy. The Shirt class exhibits the highest epistemic uncertainty (0.087) whilst Trouser exhibits the lowest (0.012). This inverse relationship between accuracy and epistemic uncertainty demonstrates that the BNN is correctly identifying the classes it struggles with, rather than producing uniformly high or low uncertainty regardless of difficulty.

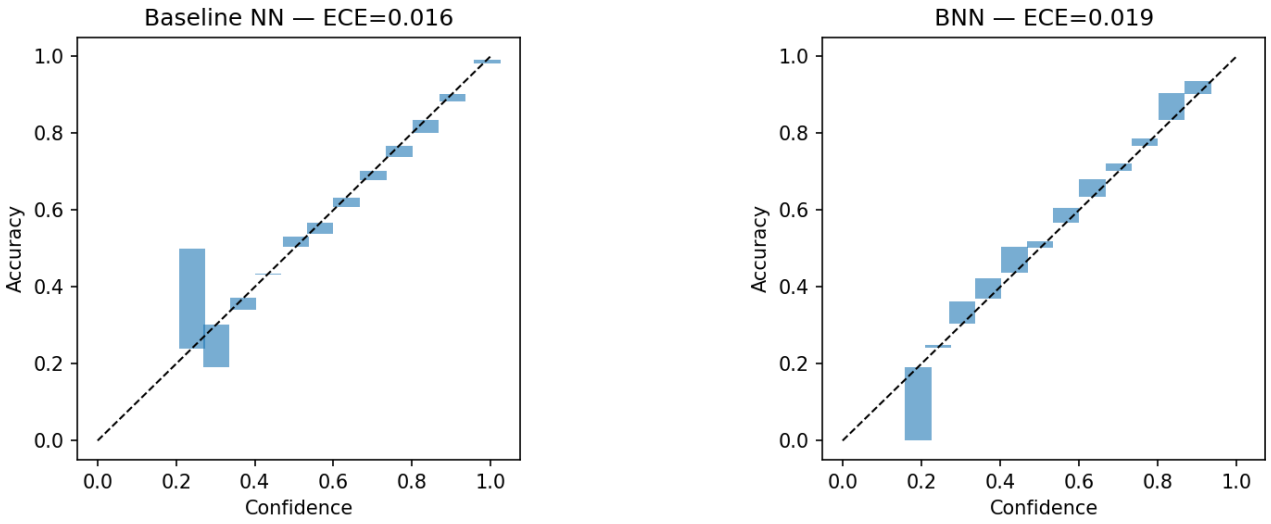
3.2 Calibration

A well calibrated model is one whose confidence scores accurately reflect its empirical accuracy, i.e. a model expressing 80% confidence should be correct approximately 80% of the time. We evaluate calibration using the Expected Calibration Error (ECE), defined as the weighted average gap between confidence and accuracy across confidence bins:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\text{acc}(B_b) - \text{conf}(B_b)\right| \tag{14}$$

where B_b is the set of samples in bin b and N is the total number of samples. A lower ECE indicates better calibration, with 0 being perfect.

Figure 2 shows the reliability diagrams for both models. A perfectly calibrated model follows the diagonal dashed line exactly.



(a) Baseline NN — ECE = 0.016

(b) BNN — ECE = 0.019

Figure 2: Reliability diagrams for the Baseline NN and BNN. Both models follow the diagonal closely, indicating good calibration in both cases.

Both models are well calibrated, with bars hugging the diagonal closely across the full confidence range. Both models achieve comparable ECE scores (0.016 and 0.019 for the baseline and BNN respectively), indicating similar calibration performance on this dataset. Interestingly, the baseline achieves a marginally lower ECE, suggesting it is slightly better calibrated than the BNN. This is likely due to the KL term dominating training, producing slightly less well calibrated confidence scores than the baseline. This is a somewhat surprising result. Standard neural networks are often criticised for being overconfident (producing high confidence predictions even when wrong). However on FashionMNIST, the baseline remains well calibrated without any explicit uncertainty mechanism. A likely explanation is that FashionMNIST is a relatively simple and clean dataset, in other words the baseline does not encounter sufficient out-of-distribution or ambiguous inputs to become systematically overconfident. On a more complex or noisy dataset, the gap between the two models would likely be more pronounced.

3.3 Uncertainty Decomposition

A key advantage of the BNN over the baseline is its ability to decompose predictive uncertainty into epistemic and aleatoric components [3], as defined in Section 2.5. Figure 3 shows the distribution of both uncertainty components across the 10,000 test images.

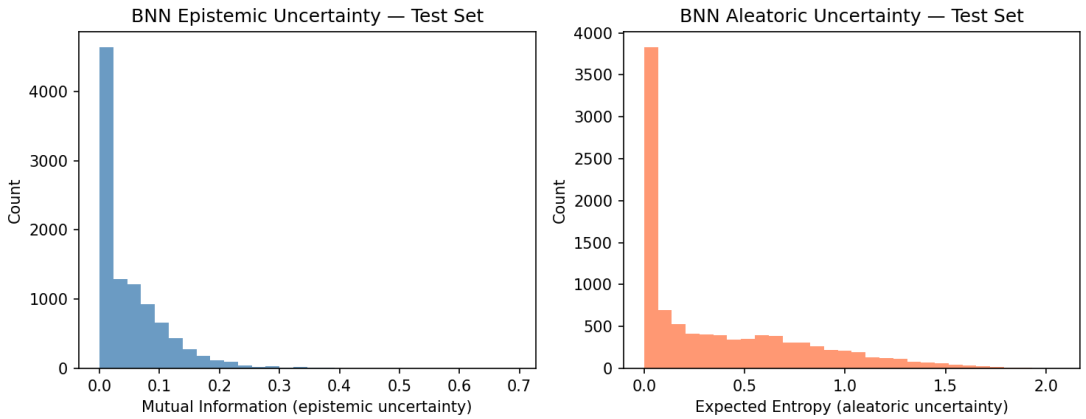


Figure 3: Distribution of epistemic uncertainty (mutual information, left) and aleatoric uncertainty (expected entropy, right) across the FashionMNIST test set.

Several observations can be drawn from Figure 3. First, epistemic uncertainty is heavily concentrated near zero where the vast majority of test images have low mutual information, indicating the model is generally confident about its weight estimates. The long tail extending to 0.7 represents a small number of genuinely ambiguous or unusual images where different weight samples produce very different predictions.

Second, aleatoric uncertainty is considerably more spread out, with values extending up to 1.5. This indicates that data ambiguity is a larger source of uncertainty than weight uncertainty for FashionMNIST. This is expected as classes such as Shirt, Pullover and Coat are visually similar and genuinely difficult to distinguish, regardless of how well the model knows its weights. No amount of additional training data would fully resolve this ambiguity, as it is inherent in the data itself. This distinction between epistemic and aleatoric uncertainty is something the baseline model is entirely unable to provide. It produces a single confidence score with no decomposition into reducible and irreducible components. This represents a meaningful advantage of the Bayesian approach, particularly in safety critical settings where understanding the source of uncertainty is as important as quantifying it.

Returning to the per-class analysis in Figure 1, the correlation between low accuracy and high epistemic uncertainty is evident. Shirt (57% accuracy, epistemic uncertainty 0.087) and Pullover (71% accuracy, 0.070) show the highest epistemic uncertainty, whilst Trouser (96% accuracy, 0.012) and Bag (95% accuracy, 0.025) show the lowest. This confirms the BNN is correctly identifying where it lacks knowledge, rather than producing uniformly high or low uncertainty across all classes.

3.4 Pros and Cons

3.4.1 Advantages

Principled uncertainty quantification. The most significant advantage of the BNN over the baseline is its ability to provide principled uncertainty estimates alongside predictions. Rather than a single confidence score, the BNN decomposes uncertainty into epistemic and aleatoric components, providing richer information about the nature of the uncertainty. This is particularly valuable in safety critical applications such as medical imaging or autonomous driving, where understanding *why* a model is uncertain is as important as knowing *that* it is uncertain.

Comparable accuracy. Despite the additional complexity of maintaining distributions over weights, the BNN achieves 86.0% test accuracy, only 2.2% below the baseline. This demonstrates that principled uncertainty quantification can be achieved with minimal cost to predictive performance on this dataset.

Regularisation via KL divergence. The KL term in the ELBO acts as a natural regulariser, penalising the posterior for deviating too far from the prior. This provides an implicit form of weight regularisation without requiring explicit techniques such as dropout or L2 weight decay, and is grounded in Bayesian theory rather than being an ad hoc addition.

3.4.2 Disadvantages

Computational overhead. A standard neural network with P parameters becomes a BNN with $2P$ learnable parameters, every weight and bias requires both a mean μ and a scale ρ, doubling memory requirements. Furthermore, obtaining uncertainty estimates at test time requires $S = 50$ forward passes per image rather than one, amounting to 500,000 forward passes for a test set of 10,000 images, which is a $50\times$ increase in inference cost that may be prohibitive in real-time applications.

Approximate inference and the mean field assumption. Variational inference provides only an approximation to the true posterior. The ELBO is a lower bound, not the exact log likelihood. In addition the variational posterior $q(\theta) = \mathcal{N}(\mu, \sigma^2)$ further assumes all weights are independent (the mean field assumption), whereas the true posterior $p(\theta|\mathcal{D})$ likely exhibits correlations between weights that this factorised Gaussian cannot capture. Together these approximations may lead to underestimation of uncertainty in practice.

Sensitivity to hyperparameter choices. The prior $p(\theta) = \mathcal{N}(0, \sigma^2_{\text{prior}})$ must be chosen before

training. A prior that is too tight imposes excessive regularisation whilst too wide a prior provides little. In this project $\sigma_{\text{prior}} = 2.0$ was chosen as a relatively uninformative prior, but this choice is somewhat arbitrary. Additionally, the KL term (5.14) dominates the NLL term (0.43) in training, suggesting the regularisation may be too strong relative to the likelihood, a known issue with mean field VI that may partly explain the accuracy gap relative to the baseline.

4 Conclusion

This project investigated the use of Bayesian Neural Networks for image classification on the Fashion-MNIST dataset. By replacing fixed point estimate weights with Gaussian variational posteriors, the BNN provides principled uncertainty quantification alongside predictions; decomposing uncertainty into epistemic and aleatoric components via Monte Carlo sampling at test time. The BNN achieved 86.0% test accuracy with an ECE of 0.019, compared to 88.2% and 0.016 for the baseline, demonstrating that meaningful uncertainty quantification can be achieved with a modest cost to predictive performance. Per-class analysis confirmed the model correctly identifies harder classes through higher epistemic uncertainty, with Shirt and Pullover showing the highest uncertainty consistent with their lower accuracy. Whilst limitations such as the mean field assumption, increased parameter count and computational cost at test time remain, this work demonstrates that Bayesian deep learning offers a compelling framework for uncertainty-aware classification in settings where understanding model confidence is as important as raw predictive performance.

References

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] Charles Blundell et al. "Weight Uncertainty in Neural Networks". In: *arXiv preprint arXiv:1505.05424* (2015).

[3] Yarin Gal. "Uncertainty in Deep Learning". PhD thesis. University of Cambridge, 2016.

[4] Han Xiao, Kashif Rasul, and Roland Vollgraf. "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms". In: *arXiv preprint arXiv:1708.07747* (2017).